



Testdokument

Projekttitel: Network Architect

Kurs: Software Engineering (DLMCSPSE01_D)

Student: Johannes Öllerer

Matrikelnummer: 92211477

Datum: 01.August 2026



Inhaltsverzeichnis

1 Einleitung und Zielsetzung	3
2 Teststrategie	3
2.1 Grundprinzipien.....	3
2.2 Vorgehen	4
2.3 Werkzeuge.....	5
3 Testumgebung.....	5
3.1 Hardwareumgebung	5
3.2 Softwareumgebung.....	6
3.3 Testausführung	6
4 Teststufen und Testarten	7
4.1 Unit-Tests	7
4.2 Integrationstests.....	8
4.3 Systemtests	9
4.4 Nichtfunktionale Tests	10
5 Testprotokoll (Testfälle).....	10
5.1 Struktur der Testfälle	10
5.2 Übersicht der Testfälle	11
5.3 Testfälle im Detail	14
6 Zusammenfassung und Bewertung.....	41
7 Verzeichnisse	43
7.1 Literaturverzeichnis.....	43
7.2 Tabellenverzeichnis	44

1 Einleitung und Zielsetzung

Die Qualitätssicherung der Anwendung „Network Architect“ verfolgt das Ziel, die korrekte Umsetzung der Spielregeln, die Stabilität der Anwendung sowie die Einhaltung der in den Anforderungs- und Spezifikationsdokumenten definierten Anforderungen sicherzustellen. Im Fokus stehen dabei insbesondere die Validierung der Hashiwokakero-inspirierten Spiellogik, die robuste Speicherung und Wiederherstellung von Spielständen sowie das zuverlässige Zusammenspiel von Model, ViewModel und View in der gewählten MVVM-Architektur.

Für das Projekt wurde eine testgetriebene Qualitätssicherung entlang einer klaren Testpyramide etabliert: Unit-Tests für isolierte Modell- und Serviceklassen, Integrationstests für das Zusammenspiel von ViewModels mit den darunterliegenden Modellen bzw. Services sowie End-to-End- bzw. UI-Tests für die Verifikation der grafischen Benutzerschnittstelle. Die Tests werden mit pytest (Version 9.0.2) unter Python 3.12.10 ausgeführt und sind in das Projekt so integriert, dass sie aus dem Projektwurzelverzeichnis per „pytest -v“ gestartet werden können.

Das vorliegende Testdokument beschreibt zunächst die angewendete Teststrategie und die verwendete Testumgebung und führt anschließend die definierten Testfälle in Form eines Testprotokolls mit ihren Ergebnissen auf. Dadurch wird der gesamte Prozess der Qualitätssicherung nachvollziehbar dokumentiert und zeigt, in welchem Umfang die identifizierten funktionalen und nicht-funktionalen Anforderungen mit Hilfe der durchgeführten Tests abgesichert wurden.

2 Teststrategie

2.1 Grundprinzipien

Das Testen von „Network Architect“ orientiert sich am Prinzip der Testpyramide nach Cohn (Cohn, 2009), die eine dreistufige Struktur aus Unit-Tests, Integrationstests und End-to-End- bzw. UI-Tests vorschlägt. Der Ansatz priorisiert eine große Anzahl schneller, isolierter Unit-Tests an der Basis der Pyramide, ergänzt durch weniger, aber gezielt eingesetzte Integrationstests auf ViewModel-Ebene sowie einige UI-orientierte Tests für die View-Schicht. Dadurch wird sichergestellt, dass die kritische Spiellogik (z.B. Validierung von Bridges, Erkennung gelöster Puzzles) vollständig automatisiert abgesichert ist, während manuelle und UI-bezogene Tests auf das notwendige Maß beschränkt bleiben.

Reine Datenklassen ohne eigene Logik (Player, Bridge, Difficulty, GridPoint) wurden bewusst nicht mit Unit-Tests versehen, da keine testbare Geschäftslogik vorhanden ist. Die View-Schicht enthält gemäß MVVM-Prinzip ebenfalls keine Geschäftslogik; ihr Verhalten wird daher primär durch ViewModel-Tests mit Qt-Signal-Verifikation sichergestellt.

2.2 Vorgehen

Zu Beginn der Implementierungsphase verfügte der Autor über begrenzte praktische Erfahrung im Schreiben von automatisierten Tests. Die ersten Unit-Tests waren funktional korrekt und inhaltlich sinnvoll strukturiert, jedoch wurden Testphasen nicht explizit durch Kommentare oder Docstrings nach dem Arrange-Act-Assert (AAA)-Muster benannt und voneinander abgegrenzt.

Im Laufe der Bearbeitung wurde das AAA-Muster als anerkanntes Strukturprinzip für Testfälle identifiziert. Es gliedert jeden Testfall in drei klar benannte Phasen: die Vorbereitung des Testzustands (Arrange), die Ausführung der zu testenden Funktion (Act) und die Überprüfung des Ergebnisses (Assert). Laut einer empirischen Studie, welche im IEEE Transactions on Software Engineering (Wei et al., 2025 S. 10-14) veröffentlicht wurde, verbessert das explizite Kennzeichnen dieser Phasen die Verständlichkeit und Wartbarkeit von Unit-Tests erheblich, da es eine einheitliche und vorhersehbare Struktur schafft. Ab diesem Zeitpunkt wurde das Muster konsequent mit entsprechender Kommentierung in allen neu geschriebenen Tests angewendet.

Die früher verfassten Tests wurden bewusst nicht nachträglich in das AAA-Muster umgeschrieben. Bei einem professionellen Softwareprojekt wäre eine solche Überarbeitung im Sinne der Code- und Testqualität selbstverständlich. In diesem Lernprojekt wurde jedoch die Entscheidung getroffen, den Lernverlauf des Autors sichtbar zu lassen: Die Gesamtheit der Testsuite dokumentiert damit nicht nur die technische Qualitätssicherung, sondern auch den persönlichen Lernprozess im Umgang mit Testmethoden – was dem Anspruch eines Software-Engineering-Portfolio-Projekts entspricht.

Fehlerfälle werden gezielt mit „`pytest.raises`“ abgesichert (z.B. `ValueError` bei ungültigen Eingaben, `TypeError` bei falschen Datentypen). Regressionstests werden bei jeder größeren Codeänderung durch vollständiges Ausführen der gesamten Testsuite sichergestellt

2.3 Werkzeuge

Als Test-Framework wird pytest (Version 9.0.2) unter Python 3.12.10 eingesetzt. Für die Integrationstests auf ViewModel-Ebene wird unittest.mock verwendet, um externe Abhängigkeiten (insbesondere die SQLite-Datenbankschicht) zu isolieren. Diese Vorgehensweise der Isolation durch Dependency Injection und Mocking ist eine bewährte Praxis im Software-Engineering: Paladugu (Paladugu, 2022) zeigt, dass Dependency Injection die Testbarkeit von Softwarekomponenten direkt verbessert, da Abhängigkeiten zur Laufzeit ausgetauscht werden können, ohne den Produktionscode zu verändern.

Die DatabaseService-Tests hingegen verwenden bewusst kein Mocking, sondern eine echte SQLite-Datenbank über das SQLite-eigene „memory:“ - Flag. Dabei wird die Datenbank vollständig im Arbeitsspeicher gehalten und existiert nur für die Laufzeit des jeweiligen Tests – jeder Test startet automatisch mit einer frischen, leeren Datenbank ohne Seiteneffekte aus vorherigen Tests. Dieses Vorgehen maximiert die Realitätsnähe der Integrationstests auf Model-Ebene, da die tatsächliche SQLite-Datenbanklogik vollständig durchlaufen wird, ohne dabei auf das Dateisystem angewiesen zu sein.

Damit mit „pytest“ alle Tests einheitlich vom Projektwurzelverzeichnis aus ausgeführt werden können, ohne dass dabei Importfehler auftreten, wurde eine zentrale „conftest.py“ im Projektwurzelverzeichnis angelegt. Da der Quellcode im Unterverzeichnis „src/“ organisiert ist, würde Python beim Ausführen der Tests standardmäßig einen „ModuleNotFoundError“ werfen, weil es nicht weiß, wo sich die Projektmodule befinden. Die „conftest.py“ wird von „pytest“ automatisch beim Start geladen und fügt das „src/“ - Verzeichnis dem Python-Suchpfad (sys.path) hinzu – damit können alle Testdateien die Projektmodule (z.B. „from models.network import Network“) ohne manuelle Pfadanpassungen importieren.

3 Testumgebung

3.1 Hardwareumgebung

Die Entwicklung und Ausführung der automatisierten Tests erfolgte auf einem Notebook mit dem Betriebssystem Windows 11 Pro, Version 25H2, in einer 64-Bit-Systemumgebung. Als Prozessor kam ein AMD Ryzen 5 5500U with Radeon Graphics mit 2.10 GHz zum Einsatz. Dem Testsystem standen 36 GB Arbeitsspeicher zur Verfügung. Die gewählte Hardware war für die Entwicklung, Ausführung und Auswertung der Tests des Projekts „Network Architect“ vollständig ausreichend.

3.2 Softwareumgebung

Die automatisierten Tests des Projekts „Network Architect“ wurden unter Python 3.12.10 ausgeführt. Als zentrales Test-Framework kam pytest in der Version 9.0.2 zum Einsatz. Für die Tests der Qt-basierten Benutzeroberfläche und signalgesteuerten Komponenten wurde zusätzlich pytest-qt in der Version 4.5.0 verwendet. Die grafische Anwendung selbst basiert auf PySide6 in der Version 6.10.1.

Für die Erstellung der ausführbaren Anwendung im Rahmen der Finalisierungsphase wurde außerdem PyInstaller in der Version 6.21.0 eingesetzt. Zur Isolierung externer Abhängigkeiten innerhalb einzelner Integrationstests wurde das in Python integrierte Modul unittest.mock verwendet. Da es sich hierbei um ein Modul der Python-Standardbibliothek handelt, ist keine separate Installation erforderlich.

Installiertes Paket	Anmerkung
Python 3.12.10	Laufzeitumgebung
PySide6 6.10.1	GUI-Framework
pytest 9.0.2	Test-Framework
pytest-qt 4.5.0	Qt-spezifische Test-Utilities
PyInstaller 6.21.0	Build-Tool (für Systemtest relevant)
unittest.mock	built-in, keine Version nötig

Tabelle 1: Softwareumgebung

3.3 Testausführung

Die Testausführung im Projekt „Network Architect“ gliedert sich in automatisierte und manuelle Tests. Die automatisierten Tests umfassen 151 Testfälle auf Unit- und Integrationsebene und werden über die Kommandozeile aus dem Projektwurzverzeichnis mit dem Befehl `pytest -v` gestartet. Voraussetzung dafür ist eine aktive virtuelle Python-Umgebung mit den in Kapitel 3.2 beschriebenen Abhängigkeiten. Der manuelle Systemtest wird gesondert an der finalen ausführbaren Anwendung durchgeführt. Voraussetzung dafür ist ein Windows 10/11-System, auf dem die .exe gestartet werden kann — eine Python-Installation oder virtuelle Umgebung ist hierfür nicht erforderlich.

Die **Unit-Tests** bilden die Basis der Testpyramide und werden vollständig automatisiert über pytest ausgeführt. Sie testen isolierte Klassen und Methoden der Model-Schicht, darunter Network, GameSession und Validator, sowie die Datenbankoperationen des DatabaseService. Die datenbankbezogenen Tests laufen dabei über eine SQLite::memory:-

Datenbank, die für jeden Testfall neu aufgebaut wird und dadurch vollständige Isolation ohne Seiteneffekte gewährleistet.

Die **Integrationstests** überprüfen das Zusammenspiel von ViewModels mit den darunterliegenden Model- und Service-Klassen und werden ebenfalls automatisiert via `pytest -v` ausgeführt. Externe Abhängigkeiten wie die Datenbankschicht werden dabei durch `unittest.mock` ersetzt, um das Verhalten der ViewModels isoliert testen zu können. Für die View-Tests, die PySide6-Komponenten instanziiieren, stellt `pytest-qt` automatisch eine aktive Qt-Anwendungsinstanz bereit, sodass keine manuelle Konfiguration erforderlich ist.

Der **Systemtest** wird manuell an der mit PyInstaller erstellten ausführbaren Anwendung (.exe) auf dem in Kapitel 3.1 beschriebenen Testsystem durchgeführt. Dabei wird das Gesamtverhalten der Anwendung unter realen Bedingungen geprüft — von der Installation bis zur vollständigen Durchführung eines Spiels. Die Ergebnisse des Systemtests werden im Testprotokoll in Kapitel 5 festgehalten.

4 Teststufen und Testarten

Die im Rahmen der Teststrategie definierten Teststufen werden im Folgenden näher beschrieben. Dabei wird erläutert, welche Aspekte der Anwendung jeweils durch Unit-Tests, Integrationstests, Systemtests und nichtfunktionale Tests abgesichert werden.

4.1 Unit-Tests

Die Unit-Tests bilden die Grundlage der Testpyramide des Projekts „Network Architect“ und dienen der isolierten Überprüfung zentraler Logikbestandteile der Anwendung. Ziel dieser Teststufe ist es, Fehler in einzelnen Klassen und Methoden frühzeitig zu erkennen, bevor sich diese in höheren Schichten wie ViewModel oder Benutzeroberfläche fortpflanzen. Der Schwerpunkt liegt dabei auf der Spiellogik sowie auf den Datenbankzugriffs- und Speicherfunktionen, die unabhängig von der grafischen Oberfläche geprüft werden können.

Ein wesentlicher Teil der Unit-Tests betrifft die Model-Klassen mit eigener Geschäftslogik. Der Validator-Test prüft dabei, ob das Erzeugen einer Bridge zwischen zwei Nodes unter Berücksichtigung der dazwischenliegenden GridPoints zulässig ist. Dabei wird insbesondere überprüft, ob die betroffenen GridPoints noch frei sind, keine unzulässigen Überschneidungen entstehen und die vorgesehenen Start- und End-Nodes korrekt miteinander verbunden werden können. Der Network-Test fokussiert auf das Hinzufügen und Entfernen von Nodes und Bridges innerhalb des Netzwerks sowie auf die Methode `find_path`, die mögliche Pfade von einem Node zum Server ermittelt. Die

Tests für `GameSession` decken die zentrale Spiellogik auf Sitzungsebene ab. Geprüft wird unter anderem, ob das Platzieren und Entfernen von `Bridges` im Spielkontext korrekt funktioniert, ob das verfügbare Budget dabei richtig berücksichtigt wird und ob die Funktionen zur Lösungsprüfung und Bewertung des Netzwerks konsistente Ergebnisse liefern.

Ein weiterer Schwerpunkt der Unit-Tests liegt auf dem `DatabaseService`, der sämtliche Datenbankzugriffe des Projekts kapselt. Die Tests verwenden eine frische `SQLite`-Datenbank pro Testlauf, sodass die `SQL`-Statements und die `Python`-Logik unter realistischen Bedingungen geprüft werden, ohne Seiteneffekte zwischen einzelnen Testfällen.

Neben den Kernklassen der Spiellogik werden auch isolierte Hilfskomponenten wie der `CoordinateMapper` über Unit-Tests geprüft. Dabei werden die Koordinatentransformation zwischen Modell- und Bildschirmkoordinaten sowie Randfälle (z. B. negative Offsets oder ungültige Zellgrößen) vollständig automatisiert verifiziert.

4.2 Integrationstests

Die Integrationstests prüfen das Zusammenspiel mehrerer Komponenten innerhalb der Anwendung und konzentrieren sich im Projekt „`Network Architect`“ vor allem auf die `ViewModel`-Schicht. Im Mittelpunkt steht dabei die Überprüfung, ob `ViewModels` korrekt mit den darunterliegenden `Model`- und `Service`-Klassen zusammenarbeiten und dabei die erwarteten Zustandsänderungen, Rückgabewerte und Signale auslösen. Dadurch wird sichergestellt, dass die Geschäftslogik nicht nur isoliert in einzelnen Klassen funktioniert, sondern auch im Zusammenspiel der beteiligten Anwendungsschichten korrekt verarbeitet wird.

Ein Schwerpunkt liegt auf den Tests des `GameViewModel`, in denen unter anderem die Initialisierung, das Zurücksetzen des Spiels, die Verarbeitung von `Bridge`-Aktionen sowie die Validierung einer Lösung geprüft werden. Ergänzt wird dies durch Tests des `StatisticsViewModel`, bei denen `Slots`, `Properties` und Signalverhalten verifiziert werden. Die Integrationstests decken damit zentrale Abläufe der Interaktion zwischen Anzeigelogik und der Spiellogik ab.

Um diese Tests gezielt und isoliert durchführen zu können, werden externe Abhängigkeiten wie die Datenbankschicht nicht direkt eingebunden, sondern mit `unittest.mock` simuliert. Diese Form des Mockings wird durch die im Projekt eingesetzte

Dependency Injection unterstützt, da Abhängigkeiten wie der `db_service` an das jeweilige `ViewModel` übergeben und für Testzwecke ersetzt werden können. Die eigentliche Datenbankintegration wird dabei nicht erneut in den Integrationstests geprüft, sondern separat durch die Tests des `DatabaseService` auf Unit-Test-Ebene abgesichert.

4.3 Systemtests

Die Systemtests dienen der Überprüfung des Gesamtverhaltens der Anwendung unter realistischen Einsatzbedingungen. Im Gegensatz zu den Unit- und Integrationstests steht hier nicht mehr die isolierte Prüfung einzelner Klassen oder Komponenten im Vordergrund, sondern die Funktionsfähigkeit der vollständig aufgebauten Anwendung aus Sicht der Benutzerin bzw. des Benutzers. Ziel dieser Teststufe ist es sicherzustellen, dass die Anwendung als Gesamtsystem stabil startet, korrekt bedient werden kann und die zentralen Abläufe des Spiels fehlerfrei unterstützt.

Der Schwerpunkt der Systemtests liegt auf typischen Nutzungsszenarien der fertigen Anwendung. Dazu gehören insbesondere das Starten der Anwendung, das Anlegen und Auswählen eines Spielers, das Laden und Spielen eines Levels, das Platzieren und Entfernen von Bridges sowie die vollständige Lösung eines Levels einschließlich der anschließenden Ergebnisanzeige. Darüber hinaus werden auch Eingabvalidierungen (z. B. bei Spielernamen), Fehlersituationen wie Regelverstöße oder unzureichendes Budget sowie die Fortschrittsverarbeitung mit Freischaltung weiterer Levels und Statistikansicht geprüft. Auf diese Weise wird das Verhalten der Anwendung nicht nur im „Happy Path“, sondern auch in relevanten Fehler- und Randfällen unter Bedingungen überprüft, die dem späteren Einsatz der finalen Version entsprechen.

Der manuelle Systemtest wird an der mit `PyInstaller` erzeugten ausführbaren Anwendung durchgeführt und erfordert daher keine separate Python-Umgebung. Ergänzend existieren automatisierte View-Tests für die `GameView`, die ausgewählte UI-nahe Interaktionen und die Schnittstelle zwischen View und `ViewModel` verifizieren. Diese View-Tests ersetzen den manuellen Systemtest jedoch nicht, sondern ergänzen ihn, da sie nur Teilaspekte der grafischen Oberfläche prüfen, während die Systemtests das Zusammenwirken aller beteiligten Komponenten innerhalb der fertigen Anwendung abdecken.

4.4 Nichtfunktionale Tests

Neben der funktionalen Korrektheit wurden im Projekt „Network Architect“ auch ausgewählte nichtfunktionale Aspekte betrachtet. Im Vordergrund standen dabei insbesondere die Stabilität der Anwendung, die grundsätzliche Bedienbarkeit der Benutzeroberfläche sowie die Ausführbarkeit der finalen Anwendung in der vorgesehenen Zielumgebung unter Windows.

Die Prüfung dieser nichtfunktionalen Eigenschaften erfolgte vor allem im Rahmen der manuellen Tests der ausführbaren Anwendung. Dabei wurde darauf geachtet, ob die Anwendung zuverlässig startet, typische Benutzeraktionen ohne Abstürze verarbeitet und die vorgesehenen Interaktionen innerhalb der grafischen Oberfläche nachvollziehbar und konsistent ausführbar sind. Zusätzlich liefern die automatisierten Tests der ViewModel- und View-Schicht indirekte Hinweise auf die Stabilität zentraler Interaktionsabläufe, ersetzen jedoch keine vollständige Usability-Bewertung.

Auf weitergehende nichtfunktionale Testarten, etwa formale Last-, Sicherheits- oder Performancetests, wurde im Rahmen dieses Projekts verzichtet. Dies ist dadurch begründet, dass es sich bei „Network Architect“ um eine lokal ausgeführte Einzelplatzanwendung mit begrenztem Systemumfang handelt und der Schwerpunkt des Portfolios auf der nachvollziehbaren Anwendung von Software-Engineering-Methoden, der Dokumentation sowie der funktionalen Qualitätssicherung liegt. Ein im Projektverlauf erkanntes Problem betrifft die Zeit, die zum Prüfen der Lösungen benötigt wird. Bei großen Levels mit vielen Bridges steigt diese stark an. Eine grundlegende algorithmische Optimierung dieser Prüfmechanismen war im gegebenen zeitlichen und inhaltlichen Rahmen des Projekts nicht umsetzbar, da der Fokus auf der funktionalen Fertigstellung und Dokumentation der Kernanforderungen lag. Die Einschränkung ist daher bewusst als offener Punkt akzeptiert.

5 Testprotokoll (Testfälle)

5.1 Struktur der Testfälle

Die im folgenden Testprotokoll aufgeführten Testfälle werden nach einem einheitlichen Schema dokumentiert. Jeder Testfall enthält eine eindeutige Testfall-ID, eine Kurzbeschreibung, die erforderlichen Vorbedingungen, die Eingabedaten beziehungsweise auszuführenden Aktionen, das erwartete Ergebnis sowie das tatsächlich beobachtete Ergebnis. Auf diese Weise wird eine nachvollziehbare und einheitliche Dokumentation der durchgeführten Tests sichergestellt. Die Testfälle werden in den nachfolgenden Abschnitten nach Teststufen gruppiert, folgen jedoch durchgängig dieser gemeinsamen Struktur.

5.2 Übersicht der Testfälle

Unit-Test

Testfall-ID	Bereich / Klasse	Kurzbeschreibung
UT-01	DatabaseService	Prüfung der Spielerverwaltung einschließlich Anlegen, Suchen und Auslesen von Spielerdaten.
UT-02	DatabaseService	Prüfung der Levelverwaltung einschließlich Erstellen, Laden, Aktualisieren und Löschen von Levels.
UT-03	DatabaseService	Prüfung der Speicherung abgeschlossener Levels sowie der Auswertung freigeschalteter und bereits gespielter Inhalte.
UT-04	Validator	Prüfung der Regeln zur Validierung von GridPoints und zulässigen Bridge-Verläufen zwischen Nodes.
UT-05	Network	Prüfung des Hinzufügens und Entfernens von Bridges sowie der Server- und Pfadermittlung im Netzwerk.
UT-06	GameSession	Prüfung zentraler Spielfunktionen wie Bridge-Platzierung, Budgetverwaltung, Lösungsprüfung und Bewertungslogik.
UT-07	Node	Prüfung grundlegender Funktionen der Node-Klasse.
UT-08	Level	Prüfung grundlegender Funktionen der Level-Klasse, insbesondere der Übernahme der Difficulty-Eigenschaften und der Erstellung des Spielfelds.
UT-09	BridgeType / NodeType	Prüfung der verwendeten Enum-Typen der Spiellogik, insbesondere BridgeType und NodeType.
UT-10	NodeConfig	Prüfung des Hinzufügens von Nodes zur Levelkonfiguration über NodeConfig.add_node sowie der Vermeidung doppelter Einträge.
UT-11	CoordinateMapper	Prüfung der Umrechnung zwischen Spielfeldkoordinaten und Koordinaten der Benutzeroberfläche.

Tabelle 2: Übersicht Unit-Test

Integrationstests

Testfall-ID	Bereich / Klasse	Kurzbeschreibung
IT-01	GameViewModel	Prüfung der Initialisierung des GameViewModel einschließlich Laden von Spieler- und Leveldaten, Aufbau des Spielfelds für die Oberfläche sowie Zurücksetzen und Zeitsteuerung
IT-02	GameViewModel	Prüfung der Verarbeitung von Aktionen zum Hinzufügen und Entfernen von Verbindungen im GameViewModel, einschließlich Eingabeprüfung, Weitergabe an die Spiellogik, Aktualisierung des Zustands und Rückmeldung bei Fehlern.
IT-03	GameViewModel	Prüfung der Abschlussprüfung im GameViewModel für gelöste und nicht gelöste Spielsituationen sowie der anschließenden Berechnung und Speicherung von Ergebnissen und der Ausgabe geeigneter Rückmeldungen.
IT-04	StatisticsViewModel	Prüfung des StatisticsViewModel hinsichtlich Initialisierung, Laden verfügbarer Spieler und Levels, Ermittlung spieler- und levelbezogener Statistikdaten sowie Bereitstellung dieser Daten und Signale für die Oberfläche.

Tabelle 3: Übersicht Integrationstests

Systemtests

Die Systemtest-Stufe umfasst sowohl den manuellen Test der ausführbaren Anwendung als auch ergänzende automatisierte View-Tests. Dadurch werden sowohl das Gesamtsystem als auch ausgewählte UI-nahe Interaktionen überprüft.

Testfall-ID	Bereich / Klasse	Kurzbeschreibung
VT-01	GameView	Prüfung der grundlegenden Initialisierung der GameView mit einem injizierten GameViewModel sowie der Verfügbarkeit zentraler Bedienelemente der Oberfläche
VT-02	GameView	Prüfung ausgewählter UI-Zustände der GameView, insbesondere des Umschaltens zwischen Arbeitsmodi sowie der Rücksetzung interner Auswahlen innerhalb der Oberfläche.
VT-03	GameView	Prüfung der Schnittstelle zwischen GameView und GameViewModel bei ausgewählten Benutzeraktionen zum Platzieren und Entfernen von Bridges.

Tabelle 4: Übersicht UI/View-Tests

Testfall-ID	Bereich	Kurzbeschreibung
ST-01	Anwendung starten	Prüfung, ob die mit PyInstaller erzeugte ausführbare Datei von „Network Architect“ auf einem Zielsystem ohne Python-Installation erfolgreich gestartet werden kann und die Startoberfläche korrekt angezeigt wird.
ST-02	Gesamtdurchlauf	Prüfung, ob ein vollständiges Spiel (vom Laden eines Levels bis zur erfolgreichen Lösung) ohne Fehler durchführbar ist und alle relevanten Spielregeln korrekt angewendet werden.
ST-03	Fortschrittspeicherung nach Levelabschluss	Prüfung, ob der Spielfortschritt eines Spielers erst nach vollständigem Abschluss eines Levels gespeichert wird und dadurch das nächste Level für diesen Spieler freigeschaltet wird.

ST-04	Robustheitstest	Prüfung der Robustheit der Anwendung bei typischen Fehlbedienungen im UI (z. B. ungültige Bridge-Platzierungen, Abbrechen von Dialogen, unerwartete Klicks)
ST-05	PyInstaller	Prüfung, ob die mit PyInstaller erzeugte .exe auch auf einem zweiten Windows-System mit abweichender Hardwarekonfiguration lauffähig ist.
ST-06	Statistikansicht	Prüfung der Anzeige spielbezogener Statistiken für den aktiven Spieler.
ST-07	Ergebnisanzeige	Prüfung des Levelabschluss-Dialogs und des angezeigten Score-Feedbacks nach erfolgreichem Abschluss.
ST-08	Levelfreischaltung	Prüfung der Aktualisierung des Fortschritts und der Freischaltung weiterer Levels nach einem Abschluss.
ST-09	Ungültige Spielernamen	Prüfung der Eingabvalidierung bei leeren, zu kurzen oder zu langen Spielernamen.

Tabelle 5: Übersicht Systemtests

5.3 Testfälle im Detail

Im Folgenden werden die in Kapitel 5.2 aufgeführten Testfälle im Detail dokumentiert. Jeder Testfall enthält eine eindeutige Testfall-ID, eine Kurzbeschreibung, die erforderlichen Vorbedingungen, die Eingabedaten beziehungsweise auszuführenden Aktionen, das erwartete Ergebnis sowie das tatsächlich beobachtete Ergebnis. Diese Struktur entspricht den formalen Anforderungen an das Testprotokoll und stellt eine einheitliche, nachvollziehbare Darstellung der Qualitätssicherung über alle Ebenen der Testpyramide hinweg sicher.

5.3.1 Unit Tests

UT-01 – Spielerverwaltung im DatabaseService

Kurzbeschreibung:

Prüfung der Spielerverwaltung einschließlich Anlegen, Suchen und Auslesen von Spielerdaten.

Vorbedingungen:

- Eine initialisierte Testdatenbank steht zur Verfügung.
- Das DatabaseService ist betriebsbereit.

Eingabedaten / Aktionen:

- Ein neuer Spieler wird angelegt.
- Der Spieler wird anschließend über seinen Namen und seine ID wieder ausgelesen.
- Zusätzlich werden leere, zu kurze, zu lange und bereits vergebene Namen geprüft.
- Außerdem wird das Auslesen mit ungültigen und nicht vorhandenen IDs beziehungsweise Namen getestet.

Erwartetes Ergebnis:

- Der Spieler wird korrekt in der Datenbank gespeichert.
- Die Such- und Leseoperationen liefern die erwarteten Spielerdaten zurück.
- Ungültige Eingaben werden mit ValueError abgewiesen.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

UT-02 – Levelverwaltung im DatabaseService

Kurzbeschreibung:

Prüfung der Levelverwaltung einschließlich Erstellen, Laden, Aktualisieren und Löschen von Levels.

Vorbedingungen:

- Ein DatabaseService steht mit einer leeren, initialisierten Datenbank zur Verfügung.
- Eine gültige Node-Konfiguration im JSON-Format liegt für erfolgreiche Testfälle vor.

Eingabedaten / Aktionen:

- Ein Level wird mit gültiger Difficulty, Zielwerten, Startbudget und gültiger Node-Konfiguration angelegt.
- Das Level wird anschließend über seine ID geladen und seine Eigenschaften werden geprüft.
- Es werden mehrere Levels erzeugt und über `get_all_levels()` abgefragt.
- Ein bestehendes Level wird verändert
- Es wird ein Level gelöscht
- Zusätzlich werden ungültige Fälle getestet

Erwartetes Ergebnis:

- Ein gültig angelegtes Level wird korrekt gespeichert und kann mit allen Metadaten sowie rekonstruierter Node-Konfiguration wieder geladen werden.
- `get_all_levels()` liefert bei leerer Datenbank eine leere Liste und bei befüllter Datenbank alle Levels in konsistenter Reihenfolge zurück.
- Änderungen werden korrekt persistiert und beim erneuten Laden sichtbar.
- Nach dem Löschen ist das betreffende Level nicht mehr abrufbar.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

UT-03 – Fortschrittsspeicherung im DatabaseService

Kurzbeschreibung:

Prüfung der Speicherung abgeschlossener Levels sowie der Auswertung freigeschalteter und bereits gespielter Inhalte.

Vorbedingungen:

- Eine frische Testdatenbank steht zur Verfügung.
- Mindestens ein Spieler und ein Level wurden angelegt.

Eingabedaten / Aktionen:

- Ein abgeschlossener Leveldurchlauf wird gespeichert.
- Die gespeicherten Daten werden über die dafür vorgesehenen Datenbankmethoden wieder ausgelesen.
- Zusätzlich werden ungültige Eingaben wie nicht existente IDs oder negative Werte geprüft.
- Es werden außerdem ungültige Fälle geprüft.

Erwartetes Ergebnis:

- Ein gespeicherter Fortschritt wird korrekt persistiert und mit allen zugehörigen Werten wieder ausgelesen.
- Die Datenbankmethoden liefern persistente Ergebnisse
- Ungültige Eingaben werden mit `ValueError` abgewiesen.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

UT-04 – Bridge-Validierung mit Validator

Kurzbeschreibung:

Prüfung der Regeln zur Validierung von GridPoints und zulässigen Bridge-Verläufen zwischen Nodes.

Vorbedingungen:

- Ein gültiges Test-Setup mit Nodes und den dazwischenliegenden GridPoints ist vorbereitet.
- Es existieren sowohl freie als auch blockierte beziehungsweise fachlich unzulässige Verbindungssituationen.

Eingabedaten / Aktionen:

- Es werden zulässige Bridge-Verbindungen zwischen zwei Nodes mit freien Zwischenpunkten geprüft
- Zusätzlich werden ungültige Konstellationen getestet, z. B. blockierte GridPoints, unzulässige Überschneidungen oder Verbindungen mit unpassenden Start- und Endpunkten.

Erwartetes Ergebnis:

- Zulässige Bridge-Verläufe werden vom Validator akzeptiert.
- Verbindungen mit belegten Zwischenpunkten, unzulässigen Überschneidungen oder ungültigen Endpunkten werden zuverlässig erkannt und abgelehnt.
- Die Validierungslogik liefert damit konsistente Ergebnisse für erlaubte und nicht erlaubte Bridge-Konstellationen.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

UT-05 – Netzwerkoperationen im Network-Modell

Kurzbeschreibung:

Prüfung des Hinzufügens von Nodes zum Netzwerk, des Hinzufügens und Entfernens von Bridges sowie der Server- und Pfadermittlung im Network-Modell.

Vorbedingungen:

- Ein Network-Objekt ist initialisiert.
- Ein Raster aus GridPoints sowie geeignete Client- und Server-Nodes stehen für die Tests zur Verfügung.

Eingabedaten / Aktionen:

- Nodes werden dem Netzwerk hinzugefügt und wieder gelöscht
- Bridges werden hinzugefügt und wieder gelöscht
- Zusätzlich werden ungültige Bridge-Operationen geprüft, z. B. blockierte Verläufe, nicht passende GridPoints oder fehlerhafte Eingabetypen.
- Für verschiedene Netzwerkzustände werden ein Pfad von einem Client zum Server sowie der vorhandene Server-Node abgefragt.

Erwartetes Ergebnis:

- Nodes und Bridges werden korrekt in den internen Netzwerkzustand übernommen und beim Entfernen wieder vollständig zurückgesetzt.
- Verbindungszähler und belegte GridPoints bleiben konsistent.
- Ungültige Bridge-Operationen werden mit einer geeigneten Ausnahme abgewiesen, ohne den Netzwerkzustand zu beschädigen.
- Die Pfadermittlung liefert für verbundene Konstellationen konsistente Pfade, und die Serverermittlung gibt den vorhandenen Server korrekt zurück beziehungsweise weist einen Zugriff ohne Server kontrolliert zurück.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

UT-06 – Spiellogik in der GameSession

Kurzbeschreibung:

Prüfung zentraler Spielfunktionen wie Bridge-Platzierung, Budgetverwaltung, Lösungsprüfung und Bewertungslogik.

Vorbedingungen:

- Eine GameSession mit gültigem Player, geladenem Level, Startbudget und konfigurierten Nodes ist vorbereitet.

Eingabedaten / Aktionen:

- Es werden gültige Bridges mit unterschiedlichen BridgeTypes platziert.
- Anschließend werden Bridges wieder entfernt.
- Zusätzlich wird ein Platzierungsversuch bei zu geringem Budget geprüft.
- Es werden vollständige und unvollständige Netzwerkkonstellationen erzeugt und mit `is_it_solved()` überprüft.
- Danach werden `calculate_redundancy_score()` und `calculate_performance()` für vorbereitete Netzwerke aufgerufen.

Erwartetes Ergebnis:

- Erfolgreich platzierte Bridges werden dem Netzwerk hinzugefügt und reduzieren das aktuelle Budget um die Kosten des jeweiligen BridgeType.
- Beim Entfernen einer Bridge wird diese aus dem Netzwerk gelöscht und das Budget korrekt rückerstattet.
- Bei unzureichendem Budget wird die Platzierung mit ValueError abgewiesen und der Zustand bleibt unverändert.
- Vollständig verbundene Netzwerke werden als gelöst erkannt, unvollständige Netzwerke dagegen nicht.
- Die Bewertungsfunktionen liefern für die vorbereiteten Testnetzwerke konsistente numerische Ergebnisse für Redundanz und Performance.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

UT-07 – Grundfunktionen der Node-Klasse

Kurzbeschreibung:

Prüfung grundlegender Funktionen der Node-Klasse.

Vorbedingungen:

- Die Klasse Node sowie die zugehörigen Node-Typen stehen zur Verfügung.

Eingabedaten / Aktionen:

- Für verschiedene Node-Typen werden Nodes erzeugt.
- Die aktuelle Verbindungsanzahl wird über entsprechende Methoden erhöht und wieder verringert.
- Dabei werden auch Grenzfälle geprüft, z. B. Erhöhung bis zur maximal zulässigen Anzahl und Verringerung bis auf 0.

Erwartetes Ergebnis:

- Eine Node übernimmt die Grenzwerte ihres Typs korrekt.
- Die Anzahl aktueller Verbindungen kann nicht über das erlaubte Maximum steigen und nicht unter 0 fallen.
- Das Verhalten der Klasse bleibt auch in Grenzfällen konsistent

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

UT-08 – Grundfunktionen der Level-Klasse

Kurzbeschreibung:

Prüfung grundlegender Funktionen der Level-Klasse, insbesondere der Übernahme der Difficulty-Eigenschaften und der Erstellung des Spielfelds.

Vorbedingungen:

- Die Klasse Level sowie der Enum Difficulty stehen zur Verfügung.

Eingabedaten / Aktionen:

- Es wird ein Level mit einer gültigen Difficulty, z. B. Difficulty.LIGHT, sowie Zielwerten und Budget erzeugt.
- Anschließend werden Difficulty, Rasterhöhe, Rasterbreite und die Größe des erzeugten Spielfelds geprüft.

Erwartetes Ergebnis:

- Das Level übernimmt Difficulty und die daraus abgeleiteten Rastermaße korrekt.
- Das Spielfeld enthält genau einen GridPoint pro Rasterzelle.
- Die Grundinitialisierung des Levels liefert damit einen konsistenten Ausgangszustand.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

UT-09 – Enum-Typen der Spiellogik

Kurzbeschreibung:

Prüfung der verwendeten Enum-Typen der Spiellogik, insbesondere BridgeType und NodeType.

Vorbedingungen:

- Die verwendeten Enum-Klassen stehen zur Verfügung.

Eingabedaten / Aktionen:

- Enum-Werte werden instanziiert beziehungsweise verwendet.
- Zugehörige Eigenschaften wie Typbezeichnungen, Kosten oder Grenzwerte werden abgefragt.

Erwartetes Ergebnis:

- Die Enums enthalten die erwarteten Werte.
- Zugehörige Eigenschaften der Enum-Einträge sind konsistent und im restlichen Modell verwendbar.
- Ungültige oder nicht definierte Werte geben einen Fehler zurück.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

UT-10 – Hinzufügen von Nodes

Kurzbeschreibung:

Prüfung des Hinzufügens von Nodes zur Levelkonfiguration über `NodeConfig.add_node` sowie der Vermeidung doppelter Einträge.

Vorbedingungen:

- Eine initialisierte `NodeConfig` liegt vor.
- `Node`, `GridPoint` und `NodeType` stehen für die Tests zur Verfügung.

Eingabedaten / Aktionen:

- Ein neuer Node wird zu einer zunächst leeren `NodeConfig` hinzugefügt.
- Anschließend wird versucht, denselben Node erneut einzufügen.

Erwartetes Ergebnis:

- Das erstmalige Hinzufügen des Nodes ist erfolgreich und der Node ist in der `NodeConfig` enthalten.
- Ein doppelter Einfügeversuch wird mit einer geeigneten Ausnahme abgewiesen.
- Die Eindeutigkeit der in der Konfiguration enthaltenen Nodes bleibt gewahrt.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

UT-11 - CoordinateMapper

Kurzbeschreibung:

Prüfung der Umrechnung zwischen Spielfeldkoordinaten und Koordinaten der Benutzeroberfläche.

Vorbedingungen:

- Die Klasse `CoordinateMapper` ist verfügbar und kann mit Standard- und benutzerdefinierten Parametern instanziiert werden.
- Die Klasse `GridPoint` steht zur Verfügung, um Grid-Koordinaten zu repräsentieren.

Eingabedaten / Aktionen:

- Es werden `CoordinateMapper`-Instanzen mit Standardwerten und mit benutzerdefinierten Parametern für `cell_size`, `offset_x` und `offset_y` erzeugt.
- Es werden `GridPoints` mit unterschiedlichen Koordinaten in Bildschirmkoordinaten umgerechnet.
- Es werden Bildschirmkoordinaten, Klicks innerhalb einer Zelle zurück in Grid-Koordinaten konvertiert.
- Es werden Bildschirmerklicks auf Zellen mit und ohne vorhandene `GridPoints` getestet, einschließlich einer leeren `GridPoint`-Liste und einer Liste mit mehreren Punkten.
- Es werden ungültige Parameter zur Initialisierung des `CoordinateMapper` verwendet.

Erwartetes Ergebnis:

- `CoordinateMapper`-Instanzen übernehmen Standard- und benutzerdefinierte Parameter korrekt; `cell_size` muss positiv sein, Offsets dürfen nicht negativ sein, andernfalls wird eine `ValueError` ausgelöst.
- `GridPoints` werden konsistent in Bildschirmkoordinaten und wieder zurück in Grid-Koordinaten umgerechnet; Klicks innerhalb derselben Zelle führen zu identischen Grid-Koordinaten.
- Beim Mapping von Bildschirmerklicks auf `GridPoints` wird der passende `GridPoint` gefunden, falls einer existiert; andernfalls liefert die Methode `None`, auch wenn die `GridPoint`-Liste leer ist oder mehrere Punkte enthalten sind.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

5.3.2 Integrationstest

IT-01 - Initialisierung, Zurücksetzen und Zeitsteuerung im GameViewModel

Kurzbeschreibung:

Prüfung der Initialisierung des GameViewModel einschließlich Laden von Spieler- und Leveldaten, Aufbau des Spielfelds für die Oberfläche sowie Zurücksetzen und Zeitsteuerung

Vorbedingungen:

- Ein GameViewModel mit vorbereiteter Spielsitzung ist initialisiert.
- Für erfolgreiche Aktionen ist ein Verbindungstyp ausgewählt.
- Die für den Test relevante Spielsitzungs-Klasse wird durch Mocks ersetzt.

Eingabedaten / Aktionen:

- Verbindungsaktionen werden mit gültigen und ungültigen Knotenkennungen und Rasterpositionen ausgelöst.
- Es werden Aktionen ausgeführt, bei denen kein Verbindungstyp gewählt wurde.
- Für erfolgreiche Fälle wird geprüft, ob die Anfrage an die Spiellogik zur Erstellung einer Verbindung weitergereicht wird.
- Für Fehlersituationen werden Ausnahmen in der Spiellogik simuliert.
- Entfernenaktionen werden mit vorhandenen und nicht vorhandenen Verbindungskennungen angestoßen und ihr Einfluss auf Budget, Knotenstatus und Rückmeldungen wird geprüft.

Erwartetes Ergebnis:

- Ungültige Knoten- oder Rasterangaben sowie Aktionen ohne gewählten Verbindungstyp führen zu einer Fehlermeldung, ohne dass sich der Spielzustand verändert.
- Zulässige Verbindungsaktionen werden an die Spiellogik weitergegeben und dort genau einmal ausgeführt.
- Nach erfolgreichen Änderungen werden Signale für geänderte Verbindungen, aktualisiertes Budget und geänderte Knotenzustände ausgelöst.
- Tritt während der Verarbeitung eine Ausnahme auf, wird eine verständliche Fehlermeldung erzeugt, die sowohl den Kontext der Aktion als auch die Ursache beschreibt.

- Beim Entfernen vorhandener Verbindungen wird die entsprechende Verbindung in der Spiellogik entfernt, das Budget angepasst und der geänderte Zustand über Signale gemeldet.
- Versuche, nicht existente Verbindungskennungen zu entfernen, lösen eine Fehlermeldung aus, ohne den Spielzustand zu verändern.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

IT-02 - Verarbeitung von Verbindungsaktionen im GameViewModel

Kurzbeschreibung:

Prüfung der Verarbeitung von Aktionen zum Hinzufügen und Entfernen von Verbindungen im GameViewModel, einschließlich Eingabeprüfung, Weitergabe an die Spiellogik, Aktualisierung des Zustands und Rückmeldung bei Fehlern.

Vorbedingungen:

- Eine Session und angebundenem Testdatenservice sind initialisiert.
- Für erfolgreiche Aktionen ist ein Verbindungstyp ausgewählt.
- Die für den Test relevante Spielsitzungs-Klasse wird durch Mocks ersetzt.

Eingabedaten / Aktionen:

- Verbindungsaktionen werden mit gültigen und ungültigen Knotenkennungen und Rasterpositionen ausgelöst.
- Es werden Aktionen ausgeführt, bei denen kein Verbindungstyp gewählt wurde.
- Für erfolgreiche Fälle wird geprüft, ob die Anfrage an die Spiellogik zur Erstellung einer Verbindung weitergereicht wird.
- Für Fehlersituationen werden Ausnahmen in der Spiellogik simuliert.
- Entfernenaktionen werden mit vorhandenen und nicht vorhandenen Verbindungskennungen angestoßen und ihr Einfluss auf Budget, Knotenstatus und Rückmeldungen wird geprüft.

Erwartetes Ergebnis:

- Ungültige Knoten- oder Rasterangaben sowie Aktionen ohne gewählten Verbindungstyp führen zu einer Fehlermeldung, ohne dass sich der Spielzustand verändert.
- Zulässige Verbindungsaktionen werden an die Spiellogik weitergegeben und dort genau einmal ausgeführt.

- Nach erfolgreichen Änderungen werden Signale für geänderte Verbindungen, aktualisiertes Budget und geänderte Knotenzustände ausgelöst.
- Tritt während der Verarbeitung eine Ausnahme auf, wird eine verständliche Fehlermeldung erzeugt, die sowohl den Kontext der Aktion als auch die Ursache beschreibt.
- Beim Entfernen vorhandener Verbindungen wird die entsprechende Verbindung in der Spiellogik entfernt, das Budget angepasst und der geänderte Zustand über Signale gemeldet.
- Versuche, nicht existente Verbindungskennungen zu entfernen, lösen eine Fehlermeldung aus, ohne den Spielzustand zu verändern.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

IT-03 – Abschlussprüfung und Ergebnisspeicherung im GameViewModel

Kurzbeschreibung:

Prüfung der Abschlussprüfung im GameViewModel für gelöste und nicht gelöste Spielsituationen sowie der anschließenden Berechnung und Speicherung von Ergebnissen und der Ausgabe geeigneter Rückmeldungen.

Vorbedingungen:

- Eine Session und angebundenem Testdatenservice sind initialisiert.
- Es werden sowohl vollständig gelöste als auch nicht gelöste Netzwerksituationen simuliert.

Eingabedaten / Aktionen:

- Die Abschlussprüfung wird in einem Zustand ausgeführt, in dem das Netzwerk als gelöst markiert ist.
- Dabei werden Status der Zeitsteuerung, Aufruf der Abschlusslogik der Spielsitzung, Berechnung von Belastungs- und Redundanzbewertung, Speicherung der Ergebnisse sowie die Rückmeldung an die Oberfläche beobachtet.
- Anschließend wird die Abschlussprüfung in einem Zustand ausgelöst, in dem das Netzwerk noch nicht gelöst ist.
- Es wird geprüft, wie sich Zeitsteuerung, Speicherung, Bewertungsberechnung und Fehlermeldungen in diesem Fall verhalten.

Erwartetes Ergebnis:

- Im gelösten Zustand wird die interne Abschlusslogik der Spielsitzung genau einmal aufgerufen.
- Die Zeitsteuerung wird angehalten und bleibt nach der erfolgreichen Prüfung deaktiviert.
- Für die gelöste Spielsituation werden eine Redundanzbewertung und eine Leistungsbewertung ermittelt und gemeinsam mit der verstrichenen Zeit im Datenservice gespeichert.
- An die Oberfläche wird eine Abschlussmeldung mit den beiden Bewertungswerten und einer formatierten Zeitangabe weitergegeben.
- Im nicht gelösten Zustand bleibt die Zeitsteuerung aktiv, es wird nichts im Datenservice gespeichert und keine Bewertung berechnet.
- Stattdessen wird eine verständliche Fehlermeldung ausgegeben, dass das Level noch nicht vollständig abgeschlossen ist.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

IT-04 – Laden und Bereitstellen von Statistikdaten im StatisticsViewModel

Kurzbeschreibung:

Prüfung des StatisticsViewModel hinsichtlich Initialisierung, Laden verfügbarer Spieler und Levels, Ermittlung spieler- und levelbezogener Statistikdaten sowie Bereitstellung dieser Daten und Signale für die Oberfläche.

Vorbedingungen:

- Ein StatisticsViewModel mit einer Testdatenbank ist initialisiert.
- Für einzelne Testfälle stehen vorbereitete Spieler, Levels und bei Bedarf abgeschlossene Levelinträge zur Verfügung.

Eingabedaten / Aktionen:

- Es wird geprüft, ob das StatisticsViewModel korrekt erzeugt wird und seine internen Listen zu Beginn leer sind.
- Die Liste der verfügbaren Spieler wird bei leerer und befüllter Datenbank abgefragt.
- Die Liste der verfügbaren Levels wird bei leerer und befüllter Datenbank abgefragt.

- Für einen Spieler mit und ohne abgeschlossene Levels werden die zugehörigen Statistikdaten geladen.
- Für ein Level mit und ohne abschließende Spieler werden die zugehörigen Statistikdaten ermittelt.
- Abschließend werden die Eigenschaften zur Rückgabe der intern gespeicherten Spielerlisten, Levellisten und Statistiken geprüft.

Erwartetes Ergebnis:

- Das StatisticsViewModel wird als gültiges Qt-Objekt erzeugt und besitzt einen funktionierenden Zugriff auf den Datenservice.
- Die internen Listen für Spieler, Levels und Statistiken sind zu Beginn leer.
- Beim Laden der verfügbaren Spieler entsteht entweder eine leere Liste oder eine Liste aus Wörterbüchern mit Kennung und Name jedes Spielers, und es wird ein Signal für geänderte Spielerliste ausgelöst.
- Beim Laden der verfügbaren Levels entsteht entweder eine leere Liste oder eine Liste aus Wörterbüchern mit Kennung und Schwierigkeitsgrad, und es wird ein Signal für geänderte Levelliste ausgelöst.
- Beim Laden spielerbezogener Statistiken ergibt sich je nach Datenlage entweder eine leere oder gefüllte Liste, und es wird ein Signal zur Anzeige der neuen Statistikdaten ausgelöst.
- Beim Laden levelbezogener Statistiken werden analog die passenden Einträge erzeugt und ein Signal zur Anzeige der Levelstatistik ausgegeben.
- Die Eigenschaften des ViewModels liefern jeweils den aktuellen Inhalt der zugehörigen internen Listen zurück.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

5.3.3 View-Tests

VT-01 – Grundlegende Initialisierung der GameView

Kurzbeschreibung:

Prüfung der grundlegenden Initialisierung der GameView mit einem injizierten GameViewModel sowie der Verfügbarkeit zentraler Bedienelemente der Oberfläche.

Vorbedingungen:

- Eine aktive Qt-Testumgebung mit QApplication steht zur Verfügung.
- Ein gemocktes GameViewModel mit Testdaten für Spielfeld, Nodes, Bridges, Budget und Bridge-Typen ist vorbereitet.
- Die für die View relevanten Signale des GameViewModel sind als Mock-Signale verfügbar.

Eingabedaten / Aktionen:

- Die GameView wird mit dem vorbereiteten GameViewModel erzeugt.
- Anschließend wird geprüft, ob die Oberfläche erfolgreich instanziiert werden kann.
- Es wird kontrolliert, ob die Bedienelemente für das Platzieren und Löschen von Bridges vorhanden sind.
- Zusätzlich wird geprüft, ob diese Bedienelemente als umschaltbare Schaltflächen ausgeführt sind.

Erwartetes Ergebnis:

- Die GameView wird fehlerfrei mit dem injizierten GameViewModel erstellt.
- Die zentralen Bedienelemente für Bridge-Platzierung und Bridge-Entfernung sind vorhanden.
- Beide Schaltflächen sind als aktivierbare beziehungsweise deaktivierbare UI-Elemente konfiguriert und damit für die vorgesehenen Arbeitsmodi geeignet.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet

VT-02 – UI-Zustände und Auswahlrücksetzung in der GameView

Kurzbeschreibung:

Prüfung ausgewählter UI-Zustände der GameView, insbesondere des Umschaltens zwischen Arbeitsmodi sowie der Rücksetzung interner Auswahlen innerhalb der Oberfläche.

Vorbedingungen:

- Eine initialisierte GameView mit gemocktem GameViewModel liegt vor.
- Die Oberfläche ist erfolgreich erzeugt und die zugehörigen UI-Elemente sind verfügbar.

Eingabedaten / Aktionen:

- Es wird geprüft, dass die Schaltflächen für Platzier- und Löschmodus anfangs nicht aktiviert sind.
- Danach wird der Löschmodus aktiviert und wieder deaktiviert.
- Zusätzlich werden interne Auswahlzustände der View vorbereitet und anschließend gezielt zurückgesetzt.
- Abschließend wird ein Rechtsklick-Ereignis simuliert, das ebenfalls eine Rücksetzung der aktuellen Auswahl auslösen soll.

Erwartetes Ergebnis:

- Die Modus-Schaltflächen befinden sich zu Beginn in einem inaktiven Ausgangszustand.
- Das Umschalten in den Löschmodus und zurück verändert den UI-Zustand erwartungsgemäß.
- Die Rücksetzlogik entfernt vorhandene interne Selektionen zuverlässig.
- Auch ein Rechtsklick auf die Zeichenfläche führt dazu, dass bestehende Auswahlzustände geleert werden.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

VT-03 – Schnittstelle zwischen GameView und GameViewModel

Kurzbeschreibung:

Prüfung der Schnittstelle zwischen GameView und GameViewModel bei ausgewählten Benutzeraktionen zum Platzieren und Entfernen von Bridges.

Vorbedingungen:

- Eine GameView mit gemocktem GameViewModel ist initialisiert.
- Das GameViewModel stellt Testdaten für Nodes, Bridges und verfügbare Verbindungstypen bereit.
- Für den Testfall zur Platzierung ist ein Verbindungstyp ausgewählt.

Eingabedaten / Aktionen:

- Es wird ein Szenario vorbereitet, in dem eine Bridge zwischen zwei Knoten hergestellt werden kann.
- Anschließend wird eine Platzierungsaktion über die Schnittstelle zwischen View und ViewModel ausgelöst.

- Zusätzlich wird ein Szenario mit bereits vorhandener Bridge vorbereitet.
- Danach wird eine Löschaktion über dieselbe Schnittstelle ausgelöst.
- Es wird jeweils geprüft, ob die zugehörige Aktion an das GameViewModel weitergereicht wurde.

Erwartetes Ergebnis:

- Aktionen zum Platzieren einer Bridge werden mit den erwarteten Parametern an das GameViewModel übergeben.
- Aktionen zum Entfernen einer vorhandenen Bridge werden ebenfalls mit der korrekten Kennung an das GameViewModel weitergereicht.
- Die GameView übernimmt damit keine eigene Spiellogik, sondern delegiert Benutzeraktionen entsprechend dem MVVM-Prinzip an das GameViewModel.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

5.3.4 Systemtests

Im Gegensatz zu den zuvor beschriebenen Unit-, Integrations- und ergänzenden View-Tests wurden die manuellen Systemtests nicht fortlaufend während der Implementierung, sondern erst in der Finalisierungsphase an der mit PyInstaller erzeugten ausführbaren .exe-Anwendung durchgeführt. Ziel dieser Teststufe war die Überprüfung des Gesamtverhaltens der finalen Anwendung unter realistischen Einsatzbedingungen auf einem Windows-System, insbesondere hinsichtlich Startverhalten, Bedienbarkeit, Spiellogik, Persistenz und Fortschrittsverarbeitung.

Da diese Teststufe erstmals das vollständige Zusammenspiel aller beteiligten Komponenten in der finalen Laufzeitumgebung prüfte, war zu erwarten, dass dabei Probleme sichtbar werden, die in den zuvor durchgeführten automatisierten Tests nicht oder nicht vollständig erkennbar waren. Aus diesem Grund ist dieses Kapitel anders aufgebaut als die vorherigen Abschnitte zu den automatisierten Tests.

Zunächst werden die Ergebnisse des ersten Systemtestdurchlaufs dokumentiert. Anschließend werden die dabei identifizierten Probleme, die analysierten Ursachen sowie die umgesetzten Korrekturen beschrieben. Abschließend werden die Ergebnisse der nach den Nachbesserungen durchgeführten Retests festgehalten, um den finalen Qualitätsstand der Anwendung nachvollziehbar darzustellen.

5.3.4.1 Ergebnisse des ersten Testdurchlaufs

Im ersten Testdurchlauf wurden die definierten Systemtests ST-01, ST-02, ST-03, ST-04, ST-05, ST-06, ST-07, ST-08 und ST-09 an der finalen .exe-Anwendung ausgeführt. Dabei zeigte sich, dass die Anwendung grundsätzlich startfähig war und mehrere Kernfunktionen bereits korrekt arbeiteten, zugleich aber in einzelnen Bereichen noch fachliche und technische Mängel bestanden.

Bestanden wurden im ersten Durchlauf insbesondere der Start der Anwendung sowie die Statistikanzeige innerhalb derselben Sitzung. Dagegen zeigten sich relevante Probleme bei der Persistenz von Spielern und Fortschritt, bei der dauerhaften Levelfreischaltung, bei der Berechnung des Redundancy Score, bei der negativen Rückmeldung für unvollständige Lösungen, bei der Löschlogik einzelner Bridges sowie bei der Eingabvalidierung ungültiger Spielernamen.

ST-01 – Installation und Start der Anwendung

Kurzbeschreibung:

Prüfung, ob die mit PyInstaller erzeugte ausführbare Datei von „Network Architect“ auf einem Zielsystem ohne Python-Installation erfolgreich gestartet werden kann und die Startoberfläche korrekt angezeigt wird.

Vorbedingungen:

- Auf dem Testsystem ist Windows 10 oder Windows 11 installiert.
- Die mit PyInstaller erzeugte .exe-Datei (Release-Build) liegt lokal im Dateisystem vor.

Eingabedaten / Aktionen:

- Die .exe-Datei wird per Doppelklick gestartet.
- Es wird gewartet, bis das Hauptfenster vollständig geladen ist.

Erwartetes Ergebnis:

- Die Anwendung startet ohne Fehlermeldung oder Crash.
- Die Startoberfläche mit Hauptmenü (z. B. Levelauswahl, Optionen, Statistiken) wird korrekt angezeigt.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

ST-02 – Durchspielen eines Levels

Kurzbeschreibung:

Prüfung, ob ein vollständiges Spiel (vom Laden eines Levels bis zur erfolgreichen Lösung) ohne Fehler durchführbar ist und alle relevanten Spielregeln korrekt angewendet werden.

Vorbedingungen:

- Die Anwendung wurde erfolgreich gestartet
- Ein gültiges Level steht zur Auswahl bereit.

Eingabedaten / Aktionen:

- Im Hauptmenü wird ein Level ausgewählt und gestartet.
- Es werden schrittweise Bridges zwischen Nodes gemäß einer bekannten korrekten Lösung platziert.
- Nach dem Platzieren der letzten korrekten Bridge wird der Levelabschluss ausgelöst.

Erwartetes Ergebnis:

- Alle zulässigen Bridge-Platzierungen werden ohne Fehlermeldung angenommen.
- Die Budgetanzeige wird bei jeder platzierten bzw. entfernten Bridge korrekt aktualisiert.
- Nach vollständiger Verbindung des Netzwerks erkennt die Anwendung den Level als gelöst und zeigt eine entsprechende Rückmeldung an.

Tatsächliches Ergebnis:

Das Setzen von Bridges funktionierte ohne Probleme. Bei aufgebrauchtem Budget erschien die korrekte Fehlermeldung. Bei einem noch nicht vollständig gelösten Level erschien auf den ersten Klick auf „Lösung prüfen“ keine eindeutige negative Rückmeldung. Die Prüffunktion akzeptierte die korrekte Lösung weiterhin zuverlässig. Beim Löschen einzelner Bridges trat teilweise zusätzlich eine Fehlermeldung auf, obwohl die Brücke grafisch korrekt entfernt wurde.

Nach den durchgeführten Korrekturen war das Verhalten wie erwartet.

ST-03 – Fortschrittsspeicherung nach Levelabschluss

Kurzbeschreibung:

Prüfung, ob der Spielfortschritt eines Spielers erst nach vollständigem Abschluss eines Levels gespeichert wird und dadurch das nächste Level für diesen Spieler freigeschaltet wird.

Vorbedingungen:

- Die Anwendung wurde erfolgreich gestartet
- Ein gültiger Spieler wurde angelegt und im Startbildschirm ausgewählt.
- Ein Level, das für diesen Spieler freigeschaltet ist, wurde gestartet.

Eingabedaten / Aktionen:

- Der Spieler löst das aktuell gestartete Level vollständig gemäß den Spielregeln (alle erforderlichen Bridges korrekt gesetzt, Budgetregeln eingehalten)
- Nach erfolgreicher Lösung wird der Levelabschluss bestätigt
- Anschließend kehrt der Spieler in den Levelauswahlbildschirm zurück.

Erwartetes Ergebnis:

- Nach Bestätigung des Levelabschlusses wird der Fortschritt des Spielers in der Datenbank gespeichert.
- In der Levelauswahl ist ersichtlich, dass das soeben absolvierte Level als abgeschlossen markiert ist.
- Das nächste Level in der vorgesehenen Reihenfolge ist nun für diesen Spieler freigeschaltet und auswählbar.

Tatsächliches Ergebnis:

Innerhalb derselben Sitzung wurden Ergebnisse korrekt angezeigt. Nach einem App-Neustart waren die zuvor angelegten Spieler jedoch verschwunden. Eine dauerhafte Speicherung der Spieler- und Fortschrittsdaten konnte daher nicht bestätigt werden. Nach den durchgeführten Korrekturen war das Verhalten wie erwartet.

ST-04 – Fehlbedienung und Robustheit

Kurzbeschreibung:

Prüfung der Robustheit der Anwendung bei typischen Fehlbedienungen im UI (z. B. ungültige Bridge-Platzierungen, Abbrechen von Dialogen, unerwartete Klicks)

Vorbedingungen:

- Ein Level ist aktiv, und mindestens eine Bridge wurde bereits korrekt platziert.

Eingabedaten / Aktionen:

- Es wird versucht, eine Bridge an einer Stelle zu platzieren, an der keine gültige Verbindung möglich ist.
- Es wird mehrfach zwischen verschiedenen Menüs und Ansichten gewechselt (z. B. Spiel, Statistiken, Hauptmenü)
- Dialoge (z. B. Speichern, Beenden) werden geöffnet und bewusst über „Abbrechen“ geschlossen.

Erwartetes Ergebnis:

- Ungültige Bridge-Platzierungen werden abgewiesen, ohne dass die Anwendung abstürzt oder in einen inkonsistenten Zustand gerät.
- Menü- und View-Wechsel funktionieren ohne Fehler; die Anwendung bleibt benutzbar.
- Abgebrochene Dialoge hinterlassen den vorherigen Zustand unverändert.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

ST-05 – Ausführung der Anwendung auf einem zweiten Testsystem

Kurzbeschreibung:

Prüfung, ob die mit PyInstaller erzeugte .exe auch auf einem zweiten Windows-System mit abweichender Hardwarekonfiguration lauffähig ist.

Vorbedingungen:

- Auf einem zweiten Testsystem ist Windows 10 oder 11 installiert.
- Die .exe-Datei wurde auf dieses System übertragen

Eingabedaten / Aktionen:

- Die .exe wird gestartet.
- Es wird ein kurzes Testspiel (Start eines Levels, Platzieren einer Bridge) durchgeführt.

Erwartetes Ergebnis:

- Die Anwendung startet ohne zusätzliche Installationsschritte.
- Die grundlegende Spielinteraktion funktioniert wie auf dem primären Testsystem.

Tatsächliches Ergebnis:

Test erfolgreich durchgeführt, Verhalten wie erwartet.

ST-06 – Statistikansicht

Kurzbeschreibung:

Prüfung der Anzeige spielbezogener Statistiken für den aktiven Spieler.

Vorbedingungen:

- Die Anwendung ist auf einem Windows-10/11-System über die mit PyInstaller erzeugte .exe gestartet.
- Ein Spieler ist verfügbar
- Mindestens ein erfolgreich abgeschlossener Level innerhalb derselben Sitzung

Eingabedaten / Aktionen:

- Öffnen der Statistikansicht innerhalb derselben Anwendungssitzung
- Kontrolle der angezeigten Ergebnisse

Erwartetes Ergebnis:

- Die Statistikansicht zeigt die Ergebnisse des aktiven Spielers nachvollziehbar an.

Tatsächliches Ergebnis:

- Innerhalb derselben Sitzung funktionierte die Statistikansicht vollständig und zeigte die Ergebnisse korrekt an.
- Die Werte waren sinnvoll und dem aktiven Spieler zuordenbar.

ST-07 – Ergebnisanzeige

Kurzbeschreibung:

Prüfung des Levelabschluss-Dialogs und des angezeigten Score-Feedbacks nach erfolgreichem Abschluss.

Vorbedingungen:

- Die Anwendung ist auf einem Windows-10/11-System über die mit PyInstaller erzeugte .exe gestartet.
- Ein gültiger Spieler ist ausgewählt und ein Level wurde erfolgreich geladen.
- Das Level kann mit den vorhandenen Spielregeln (Budget, Bridges, Verbindung aller relevanten Nodes) vollständig gelöst werden.

Eingabedaten / Aktionen:

- Das Level wird regelkonform gespielt, bis alle erforderlichen Verbindungen hergestellt sind und die Spiellogik den Zustand „Level gelöst“ meldet.
- Nach der Lösung wird die Abschlussaktion ausgelöst (z.B. durch Klick auf „Level abschließen“ oder durch automatisches Erkennen der Lösung).
- Der angezeigte Ergebnisdialog wird vollständig betrachtet, einschließlich der angezeigten Score-Werte und Zeitangaben.

Erwartetes Ergebnis:

- Nach erfolgreichem Abschluss des Levels wird ein Ergebnisdialog angezeigt, der ohne Fehlermeldungen geöffnet wird.
- Der Dialog zeigt nachvollziehbare Bewertungsinformationen an (z.B. Score, Redundanz-/Performance-Bewertung, benötigte Zeit), die mit den im Spielverlauf erzielten Leistungen übereinstimmen.

Tatsächliches Ergebnis:

Das Ergebnisfenster wurde angezeigt. Performance und Zeit wurden mit sinnvollen, plausiblen Werten passend zum gespielten Durchlauf angezeigt. Der Redundancy Score war fachlich auffällig und wirkte zu hoch.

Nach den durchgeführten Korrekturen war das Verhalten wie erwartet.

ST-08 – Levelfreischaltung

Kurzbeschreibung:

Prüfung der Aktualisierung des Spielfortschritts und der Freischaltung weiterer Levels nach erfolgreichem Abschluss eines Levels.

Vorbedingungen:

- Die Anwendung ist gestartet.
- Ein Spieler ist angelegt und als aktiver Spieler ausgewählt.

- Mindestens zwei Levels sind im System vorhanden; das erste Level ist für den Spieler verfügbar, das nächste Level ist vor Testbeginn noch nicht freigeschaltet.

Eingabedaten / Aktionen:

- Der aktive Spieler startet das erste verfügbare Level über die Levelauswahl.
- Das Level wird regelkonform vollständig gelöst, sodass die Spiellogik den Zustand „Level abgeschlossen“ meldet und der Abschlussdialog angezeigt wird.
- Nach dem Abschluss kehrt der Spieler zur Levelauswahl zurück.
- Es wird geprüft, welche Levels jetzt als verfügbar bzw. gesperrt markiert sind.
- Optional wird ein weiteres Level abgeschlossen, um zu prüfen, ob der Fortschritt mehrfach korrekt fortgeschrieben wird.

Erwartetes Ergebnis:

- Nach dem erfolgreichen Abschluss des Levels wird der Fortschritt des aktiven Spielers in der Datenbank aktualisiert; der abgeschlossene Level wird als „erledigt“ vermerkt.
- In der Levelauswahl ist der soeben abgeschlossene Level weiterhin als gespielt erkennbar, und das nächste Level in der Reihenfolge ist für diesen Spieler neu freigeschaltet.
- Nicht freigeschaltete Levels bleiben weiterhin gesperrt, wenn die dafür notwendigen vorherigen Levels noch nicht abgeschlossen wurden.
- Wird ein weiteres Level abgeschlossen, wird der Fortschritt entsprechend erweitert; es kommt zu keinen widersprüchlichen Zuständen.

Tatsächliches Ergebnis:

Im Ergebnisfenster konnten „Hauptmenü“, „Weiter spielen“ und „Nächstes Level“ ausgewählt werden. Nach Auswahl von „Hauptmenü“ wurde das Ergebnis korrekt in der Statistik angezeigt. In der Levelauswahl war Level 2 jedoch weiterhin nicht auswählbar. Nach Auswahl von „Nächstes Level“ wurde das nächste Level zwar direkt gestartet, nach Rückkehr zur Levelauswahl blieb Level 2 dort aber weiterhin gesperrt. Nach den durchgeführten Korrekturen war das Verhalten wie erwartet.

ST-09 – Ungültige Spielernamen

Kurzbeschreibung:

Prüfung der Eingabevalidierung bei leeren, zu kurzen oder zu langen Spielernamen im Startbildschirm.

Vorbedingungen:

- Die Anwendung ist gestartet.
- Die Startoberfläche mit dem Eingabefeld für den Spielernamen und der Schaltfläche zum Anlegen bzw. Auswählen eines Spielers ist sichtbar.

Eingabedaten / Aktionen:

- Im Eingabefeld wird ein leerer Spielernamen eingegeben (z.B. direktes Bestätigen ohne Eingabe).
- Es wird ein Spielernamen eingegeben, der kürzer ist als die minimale erlaubte Länge.
- Es wird ein Spielernamen eingegeben, der länger ist als die maximal erlaubte Länge.
- Bei jedem dieser Eingaben wird der Anlegen-/Bestätigen-Button ausgelöst.

Erwartetes Ergebnis:

- Leere Spielernamen werden abgewiesen; die Anwendung legt keinen Spieler an und zeigt eine verständliche Fehlermeldung oder Validierungsmeldung an.
- Zu kurze Spielernamen werden abgewiesen; der Spieler wird nicht gespeichert, und die Fehlermeldung weist auf die Mindestlänge hin.
- Zu lange Spielernamen werden ebenfalls abgewiesen; der Spieler wird nicht angelegt, und die Fehlermeldung weist auf die maximale Länge hin.

Tatsächliches Ergebnis:

Bei leerem Eingabefeld verschwand der Eingabedialog, es wurde jedoch kein Spieler angelegt. Bei Eingabe von „A“ erschien die Fehlermeldung „Faild to create player: Player name must be 2-20 character“. Ein Name mit genau 20 Zeichen wurde erfolgreich angelegt. Ein Name mit mehr als 20 Zeichen wurde mit derselben Fehlermeldung wie ein zu kurzer Name abgewiesen.

Nach den durchgeführten Korrekturen war das Verhalten wie erwartet.

5.3.4.2 Umgesetzte Korrekturen

Die im ersten Systemtestdurchlauf identifizierten Abweichungen wurden im Anschluss systematisch analysiert und schrittweise behoben. Dabei zeigte sich, dass die Probleme nicht auf einen einzelnen Fehler beschränkt waren, sondern unterschiedliche Bereiche der Anwendung betrafen, insbesondere die Spiellogik, die Eingabevalidierung, die Interaktion zwischen View und ViewModel sowie die Konfiguration der finalen .exe-Laufzeitumgebung.

Ein zentraler fachlicher Fehler betraf die Lösungsprüfung und die Berechnung des Redundancy Score. Die Ursache lag darin, dass in der internen Testkopie eines Levels die erwartete Node-Menge fälschlich direkt an den aktuellen Zustand des kopierten Netzwerks gebunden wurde. Dadurch wurde bei Redundanzprüfungen nicht mehr stabil gegen die ursprüngliche vollständige Leveldefinition geprüft, sondern teilweise gegen ein bereits verändertes Teilnetz. Die entsprechende Logik in `create_copy()` wurde daher so angepasst, dass die `node_config` der Testkopie nicht mehr direkt aus `test_session.network.nodes` übernommen wird, sondern auf Basis der ursprünglichen Leveldefinition neu aufgebaut wird. Dadurch blieb die Soll-Struktur des Levels in der Testkopie stabil, die Lösungsprüfung erkannte unvollständige Netzwerke wieder korrekt, und auch der Redundancy Score lieferte anschließend fachlich plausible Werte.

Ein weiterer Fehler betraf das Entfernen von Bridges im Spielcanvas. Die Analyse zeigte, dass eine grafisch dargestellte Bridge intern aus mehreren `QGraphicsLineItem`-Segmenten besteht, die dieselbe `bridge_id` tragen. Dadurch konnte es vorkommen, dass bei einem einzelnen Löschklick dieselbe Bridge-ID mehrfach an die Löschlogik übergeben wurde, obwohl die Bridge bereits entfernt worden war. Die View-seitige Klickauswertung wurde deshalb angepasst, sodass im Löschmodus pro Klick nur noch genau das am Klickpunkt liegende Scene-Item ausgewertet wird. Nur wenn dieses Item tatsächlich als Bridge gekennzeichnet ist, wird genau eine Bridge-ID ausgelesen und einmalig an `remove_bridge()` übergeben. Auf diese Weise wurde die zuvor auftretende zusätzliche Fehlermeldung beim korrekten grafischen Entfernen einer Bridge beseitigt.

Auch die Eingabevalidierung bei der Spielererstellung wurde überarbeitet. Im ersten Testdurchlauf zeigte sich, dass leere Eingaben nicht konsistent behandelt wurden, weil der Dialog geschlossen werden konnte, ohne eine verständliche Rückmeldung anzuzeigen. Zusätzlich war die Fehlermeldung für ungültige Namen sprachlich fehlerhaft formuliert. Die Validierungslogik wurde deshalb so angepasst, dass leere, zu kurze und

zu lange Namen einheitlich als ungültige Eingaben erkannt und mit einer verständlichen Rückmeldung behandelt werden, ohne den Dialog kommentarlos zu schließen.

Besonders relevant für die finale .exe-Anwendung waren die Korrekturen im Bereich Persistenz und Build-Konfiguration. Im ersten Systemtest zeigte sich, dass nach einem Neustart angelegte Spieler und gespeicherte Fortschritte nicht mehr vorhanden waren. Die weitere Analyse machte deutlich, dass hierfür nicht die fachliche Spiellogik allein verantwortlich war, sondern insbesondere die Laufzeitkonfiguration der ausgelieferten Anwendung. Deshalb wurden Build- und Pfadprobleme rund um Ressourcen und Datenbanken überprüft und überarbeitet, damit sich die Laufzeitumgebung der finalen .exe konsistent zur Entwicklungsumgebung verhält.

Im Zuge dieser Überarbeitung wurde die Trennung der Datenbestände klarer gefasst: Die Seed-Datenbank dient ausschließlich als Ausgangsbasis für statische Projektdaten wie mitgelieferte Level, während benutzerspezifische Laufzeitdaten getrennt davon in einer persistenten User-Datenbank gespeichert werden sollen. Für die finale Anwendung bedeutet das konzeptionell, dass Entwicklungsdatenbank, Seed-Datenbestand und nutzerspezifische Schreibdaten nicht vermischt werden dürfen, damit Fortschritt und Spielerinformationen nach einem Neustart erhalten bleiben. Entsprechend wurden die Laufzeitpfade und die Einbindung der Datenbankdateien im Build so überarbeitet, dass Ressourcen und Seed-Datenbank korrekt mitgeliefert werden und die persistenten Benutzerdaten in einer separaten user_DB.db geführt werden können. Der anschließende Retest wurde bewusst mit gelöschter user_DB.db, frischem Erststart, neuer Spieleranlage, Levelabschluss und erneutem Neustart durchgeführt, um die korrigierte Persistenz unter realistischen Bedingungen erneut zu verifizieren. Da die Levelfreischaltung fachlich eng an die korrekte Speicherung des Spielfortschritts gekoppelt ist, wurde nach den Korrekturen an Persistenz und Build auch dieser Bereich erneut geprüft. Ziel war dabei sicherzustellen, dass ein erfolgreich abgeschlossenes Level nicht nur innerhalb derselben Sitzung wirkt, sondern auch in der Levelauswahl dauerhaft als freigeschaltet erscheint und dieser Zustand nach einem Neustart bestehen bleibt.

Die Korrekturen wurden damit bewusst nicht nur auf Ebene einzelner Symptome vorgenommen, sondern sowohl in der Anwendungslogik als auch in der Laufzeitkonfiguration der finalen .exe umgesetzt. Die nachfolgenden Retests dienten dazu, diese Änderungen erneut unter realen Einsatzbedingungen zu überprüfen und den finalen Qualitätsstand der Anwendung für Phase 3 nachvollziehbar zu dokumentieren.

5.3.4.3 Ergebnisse des zweiten Systemtestdurchlaufs

Nach Umsetzung der in Kapitel 5.3.4.2 beschriebenen Korrekturen wurden die im ersten Systemtestdurchlauf auffälligen bzw. nicht vollständig bestandenen Testfälle erneut an der aktualisierten finalen .exe-Anwendung überprüft. Dabei wurden insbesondere die zuvor betroffenen Bereiche Lösungsprüfung, Berechnung des Redundancy Score, Löschlogik von Bridges, Eingabevalidierung bei der Spielererstellung, Persistenz von Spielern und Fortschritt sowie die dauerhafte Levelfreischaltung erneut getestet.

Im zweiten Testdurchlauf konnten die im ersten Durchlauf dokumentierten Fehlverhalten nicht mehr reproduziert werden. Die Anwendung reagierte bei unvollständigen Lösungen nun mit der vorgesehenen negativen Rückmeldung, der Redundancy Score wurde fachlich plausibel angezeigt, und das Entfernen einzelner Bridges funktionierte ohne die zuvor beobachtete zusätzliche Fehlermeldung. Auch die Eingabevalidierung bei ungültigen Spielernamen arbeitete nach der Überarbeitung konsistent und benutzerverständlich.

Darüber hinaus wurde die Persistenz der finalen .exe erneut unter realistischen Bedingungen geprüft. Nach frischem Start, Spieleranlage, erfolgreichem Abschluss eines Levels sowie vollständigem Schließen und erneutem Start der Anwendung blieben sowohl der angelegte Spieler als auch der gespeicherte Fortschritt und die Statistik erhalten. In diesem Zusammenhang wurde ebenfalls bestätigt, dass nach erfolgreichem Abschluss von Level 1 das nächste Level in der Levelauswahl freigeschaltet und auch nach einem Neustart weiterhin auswählbar war.

Damit konnten die im ersten Systemtestdurchlauf auffälligen Testfälle nach der Nachbesserung erfolgreich verifiziert werden. Die Retests bestätigen, dass die im Rahmen der Finalisierungsphase umgesetzten Korrekturen wirksam waren und die finale Anwendung in den zuvor betroffenen Bereichen den erwarteten Funktionsumfang erfüllt.

6 Zusammenfassung und Bewertung

Die im Projekt Network Architect durchgeführten Unit-, Integrations-, View- und Systemtests zeigen insgesamt, dass die wesentlichen funktionalen Anforderungen der Anwendung in den relevanten Teststufen überprüft wurden. Der Schwerpunkt der automatisierten Tests lag auf der Absicherung der Spiellogik, der Datenbankoperationen sowie des Zusammenspiels von Model, ViewModel und View innerhalb der gewählten

MVVM-Architektur. Ergänzend dazu wurde die lauffähige Gesamtanwendung im Rahmen manueller Systemtests an der mit PyInstaller erzeugten .exe unter realistischen Einsatzbedingungen geprüft.

Die Ergebnisse der automatisierten Tests fielen insgesamt stabil aus. Insbesondere die zentralen Regelmechaniken des Spiels, die Persistenzlogik auf Ebene der testnah geprüften Datenbankoperationen sowie die Interaktion der ViewModels mit den zugrunde liegenden Modell- und Serviceklassen konnten erfolgreich verifiziert werden. Auch die ergänzenden View-Tests zeigten, dass ausgewählte UI-nahe Interaktionen und die Schnittstelle zwischen View und ViewModel im vorgesehenen Rahmen korrekt funktionieren.

Im manuellen Systemtest zeigte sich im ersten Testdurchlauf dagegen ein differenzierteres Bild. Die Anwendung war bereits startfähig, und mehrere Kernfunktionen wie der grundlegende Spielablauf, die Statistikanzeige innerhalb derselben Sitzung sowie einzelne Eingabe- und Bedienabläufe funktionierten bereits grundsätzlich. Gleichzeitig traten jedoch in mehreren Bereichen noch relevante Abweichungen auf, insbesondere bei der Persistenz von Spielern und Fortschritt nach einem Neustart, bei der dauerhaften Levelfreischaltung, bei der Berechnung des Redundancy Score, bei der Rückmeldung für unvollständige Lösungen, bei der Löschlogik einzelner Bridges sowie bei der Konsistenz der Eingabevalidierung für Spielernamen.

Diese Befunde wurden im Anschluss analysiert und technisch nachgebessert. Die umgesetzten Korrekturen betrafen sowohl die fachliche Spiellogik als auch die View-seitige Interaktionslogik und die Konfiguration der Laufzeitumgebung der finalen .exe, insbesondere im Zusammenhang mit Build-, Pfad- und Persistenzverhalten. Die anschließenden Retests zeigten, dass die zuvor dokumentierten Fehlverhalten nicht mehr reproduzierbar waren. Damit konnten die im ersten Durchlauf auffälligen Systemtests nach der Nachbesserung erfolgreich verifiziert werden.

Insgesamt lässt sich die Qualitätssicherung des Projekts damit als nachvollziehbar und für den gegebenen Projektumfang angemessen bewerten. Besonders positiv ist, dass nicht nur erfolgreiche Tests dokumentiert wurden, sondern auch erkannte Probleme, deren Ursachen, die vorgenommenen Korrekturen und die anschließende erneute Überprüfung transparent festgehalten sind. Gerade im Kontext eines Software-Engineering-Portfolios ist dies aussagekräftiger als eine rein oberflächliche Darstellung

ausschließlich bestandener Testfälle, da der tatsächliche Engineering-Prozess mit Fehleranalyse, Nachbesserung und Verifikation sichtbar wird.

Trotz des insgesamt positiven Ergebnisses bestehen Grenzen der vorliegenden Qualitätssicherung. Die Systemtests wurden bewusst manuell durchgeführt und nicht automatisiert, was bei einer GUI-basierten Desktop-Anwendung im Rahmen des Projekts vertretbar ist, jedoch eine geringere Wiederholbarkeit mit sich bringt als automatisierte Testverfahren. Ebenso wurden weitergehende nichtfunktionale Prüfungen, etwa umfassende Performance-, Usability- oder Kompatibilitätstests auf mehreren Zielsystemen, nur begrenzt berücksichtigt.

Einige im Anforderungs- und Spezifikationsdokument definierte Funktionen, insbesondere das Tutorial-System und die Hinweis-Funktion, wurden in Phase 3 bewusst nicht umgesetzt und als technische Schuld dokumentiert; entsprechend existieren für diese Bereiche derzeit keine automatisierten Tests, sondern sie sind als potenzielle Erweiterungen zukünftiger Versionen von „Network Architect“ vorgesehen.

Für den Kontext dieses Projekts ist die erreichte Testabdeckung dennoch als geeignet zu bewerten. Die Teststrategie orientiert sich nachvollziehbar an den relevanten Teststufen, kritische Teile der Spiellogik wurden automatisiert abgesichert, und die zentralen Nutzungsszenarien der finalen Anwendung wurden zusätzlich durch manuelle Systemtests einschließlich Retests überprüft. Das Testdokument leistet damit einen wesentlichen Beitrag zur nachvollziehbaren Dokumentation der Qualitätssicherung im Projekt Network Architect.

7 Verzeichnisse

7.1 Literaturverzeichnis

- Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley.
- Wei, C. et al. (2025). *How Do Developers Structure Unit Test Cases? An Empirical Study of the AAA Pattern*. IEEE Transactions on Software Engineering.
- Paladugu, S. (2022). *Dependency Injection and Its Impact on Code Reusability and Testability*. Zenodo.

7.2 Tabellenverzeichnis

Tabelle 1: Softwareumgebung.....	6
Tabelle 2: Übersicht Unit-Test	11
Tabelle 3: Übersicht Integrationstests.....	12
Tabelle 4: Übersicht UI/View-Tests.....	13
Tabelle 5: Übersicht Systemtests	14